

Mail Parser Platform

Full System Documentation

Version: 1.0
Date: October 2025

Table Of Contents

Mail Parser Platform Full System Documentation.....	1
1. Executive summary (client-friendly).....	3
2. System architecture (high level).....	3
3. Project structure.....	4
Backend.....	4
Frontend	4
4. Environment & credentials (complete <code>.env</code> guidance)	5
5. Backend — setup & run.....	5
6. Frontend — setup & run	6
7. Gmail API (OAuth2 — Refresh token flow you used)	7
8. API Reference — accurate & complete (derived from source)	7
/auth/login — Authenticate user	8
/users/* — User management (protected)	8
/database/* — Database connections (lowdb-stored config).....	9
/files/* — Processed & incoming files	9
/imports/* — Import jobs	10
/imap/poll — IMAP manual poll trigger.....	10
/mail/fetch-mails — ad-hoc IMAP run with supplied credentials	11
/inboxes/* — mailbox management & gmail config.....	11
/mappings/* — mapping definitions	12
/archives/* — archive listing & download	13
9. Data model (lowdb) & skeleton	13
10. Workflows — end-to-end behavior	14
A. Automatic inbox polling (recommended)	14
B. Manual import from UI.....	14
11. Deployment & production tips	14
12. Monitoring, logging & maintenance	15
13. Troubleshooting checklist (quick actionable).....	15
14. Appendix.....	16
A. Full sample <code>.env</code> (put in root of backend).....	16
B. lowdb example snippet (small).....	16

1. Summary

The Mail Parser Platform is a full-stack application that automates the process of receiving emails, extracting structured attachments (such as CSV files), validating/mapping them, and inserting the results into downstream targets (databases, systems). It includes:

- A **web UI** (Next.js / React 19) for configuration and monitoring. The UI was supplied as `mail-parser-app.zip` (you uploaded).
- A **backend service** (`mail-parser-platform.zip`) built on **Node.js + Express**, which runs IMAP fetchers, parsing, mapping, and import jobs. The backend uses **lowdb** as a light JSON database for configuration & metadata; actual import destinations can be external databases defined by stored connection entries.

Business benefits:

- Reduces manual CSV handling and human errors.
- Centralizes mailbox-to-database mappings and history.
- Auditable processing with logs and archives.

2. System architecture (high level)

- **Frontend (UI)** — Next.js 15 + React 19 TypeScript app. Calls backend REST API (`/api/*` endpoints) to perform actions (create inbox, test Gmail connection, start import, view files, manage mappings).
- **Backend** — Node.js + Express REST service. Responsible for:
 - Storing configuration and metadata in a **lowdb** JSON file.
 - Connecting to Gmail using OAuth2 refresh
 - Downloading attachments, parsing them (CSV), applying mappings, and writing results to configured targets.
 - Exposing endpoints used by the UI and by cron/manual triggers.
- **Storage** — attachments and processed file copy in `uploads/` and `storage/archives/`.
- **Logging** — Winston logger with file rotation (config under `config/winston.js`).

Components communicate via HTTPS REST. The system expects the frontend to call the backend REST API at `NEXT_PUBLIC_API_URL`.

3. Project structure

Backend

Key files/folders

- `app.js` — main express app entry
- `routes/` — express routes:
 - `index.route.js` — mounts the subroutes (maps to `/auth`, `/users`, `/files`, `/database`, `/mappings`, `/imports`, `/imap`, `/mail`, `/inboxes`, `/archives`)
 - `auth.route.js`, `user.route.js`, `files.route.js`, `dbConnections.route.js`, `mappings.route.js`, `imports.route.js`, `imap.route.js`, `mail.route.js`, `inboxes.route.js`, `archive.route.js`
- `controllers/` — controller functions implementing the route logic
- `services/` — core modules:
 - `mail-imap.js` — IMAP logic (fetch, process) and `processMailWithImap` service
 - `processor.js` — CSV preview/parse and import helper functions
 - `knex-connection.js` — used for optional DB connection logic (when importing into external DBs)
- `config/lowdb.js` — lowdb setup (JSON file storage)
- `middlewares/` — authentication middleware, request logger, Joi error handling
- `uploads/`, `storage/archives/` — file storage for attachments & archives
- `db/` — lowdb JSON file (default path set in `process.env.LOWDB_FILE`)

Frontend

Key folders:

- `app/` — Next.js route-based pages (dashboard, inboxes, mappings, imports, users, etc.)
- `components/` — UI components
- `lib/` — helper functions & API wrappers (calls to `NEXT_PUBLIC_API_URL`)
- `public/` — static assets
- `styles/` — Tailwind/Tokens

4. Environment & credentials (complete .env guidance)

Backend .env

```
APP_PORT=4000

BASE_UPLOAD_DIR=./uploads

POLL_INTERVAL_SECONDS=30

TOKEN_SECRET_KEY=mySuperSecretKey123!

UPLOADS_BASE=./uploads/{mailbox-name}

CLIENT_URL=http://localhost/3000

GMAIL_CLIENT_ID=360438430623-
0a7hemmlmlaalgaifu44s1kifd8r5t0i.apps.googleusercontent.com

GMAIL_CLIENT_SECRET=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

GMAIL_REDIRECT_URI=http://localhost:4000/api/auth/google/callback

(see section 7 on gmail setup to get credetials)
```

Frontend .env

```
NEXT_PUBLIC_API_URL=http://localhost:4000/api
```

5. Backend — setup & run

Prereqs

- Node.js 18+ (the project uses modern ESM imports)
- pnpm or npm
- (Optional) Docker if you want containerized run
- If using Gmail OAuth, you must have OAuth client credentials and a refresh token (see section 7)

Install & run

1. Unzip mail-parser-platform.zip into folder mail-parser-platform.
2. cd mail-parser-platform
3. npm install
4. Create .env like above. If you rely on LOWDB_FILE, ensure parent directory exists and is writable.

5. Start:

- Dev: `npm start`
- Prod: `node app.js &`
- Or run with PM2: `pm2 start app.js --name mail-parser`

Database migrations

- **Not used** —The app uses **lowdb** (JSON file store). There are some helper functions for connecting to external DBs (via Knex) but this are for the external databases receiving the imports.

Important: file storage permissions

- Ensure `uploads/` and `storage/archives/` directories exist and are writable by the service user.

6. Frontend — setup & run

Install & run

1. Unzip `mail-parser-app.zip` to `mail-parser-app`.
2. `cd mail-parser-app`
3. `pnpm install` (preferred) or `npm install`
4. Create `.env` or ensure it exists
5. `NEXT_PUBLIC_API_URL=http://localhost:4000`
6. Start dev server: `pnpm dev` (or `npm run dev`)
7. Open `http://localhost:3000`

Integration

- UI calls the backend endpoints listed in section 8. The UI expects responses in the shapes returned by the controllers (examples below). If you host backend behind a different domain, update `NEXT_PUBLIC_API_URL`.

7. Gmail API

1. **Go to Google Cloud Console** → create or select your project.
2. **Enable Gmail API**
 - Navigate to **APIs & Services** → **Library**, search “**Gmail API**”, and click **Enable**.
3. **Set up OAuth Consent Screen**
 - Go to **APIs & Services** → **OAuth consent screen**.
 - Choose **External** if anyone can connect, or **Internal** if restricted to your organization.
 - Add required scopes (minimum):
 - `https://www.googleapis.com/auth/gmail.readonly`
 - `https://www.googleapis.com/auth/userinfo.email`
 - `https://www.googleapis.com/auth/userinfo.profile`
 - Complete all required fields and publish the app.
4. **Create OAuth Client ID**
 - Go to **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth Client ID**.
 - Choose **Application Type: Web application**.
 - Add **Authorized redirect URIs** — include your app’s callback endpoint, e.g.
 - `https://yourdomain.com/api/auth/google/callback`

(For local testing, add:

`http://localhost:4000/api/auth/google/callback`)

5. **Copy your credentials**
 - Copy the generated **Client ID** and **Client Secret** into your `.env` file as:
 - `EMAIL_CLIENT_ID=your_client_id`
 - `EMAIL_CLIENT_SECRET=your_client_secret`
 - `EMAIL_REDIRECT_URI=https://yourdomain.com/api/auth/google/callback`
6. **Connect the mailbox inside your app**
 - From the **Inboxes** section, click the  **Settings icon** next to an inbox.
 - Click “**Connect Gmail**” — this opens Google’s login and consent screen.
 - Once authorized, the refresh token is automatically stored in your JSON DB, and the inbox becomes **Active**.
7. **Test the connection**
 - Use the **Test Connection** button to verify the Gmail API connection for that inbox.
 - If successful, it will display the connected email address.

8. API Reference

Base URL: `http://<HOST>:<PORT>` (default `http://localhost:4000`)

Authentication: Only the `/users/*` routes are protected by `isAuthenticated` middleware in the code. Auth occurs via `/auth/login` which returns a JWT.

Below are the actual routes present in `routes/` with full mounted prefixes as in `index.route.js`. I list method, path, purpose, expected request fields, and representative success/error responses. Where code shows exact behavior I reproduce it.

`/auth/login` — Authenticate user

- **Method:** POST
- **Description:** Log in with email/password; returns JWT token on success.
- **Request body:**

```
{
  "email": "user@example.com",
  "password": "plainPassword"
}
```

- **Success (200):**

```
{
  "success": true,
  "token": "<jwt-token>",
  "email": "user@example.com"
}
```

- **Errors:**
 - 401 — invalid credentials
 - 500 — server error

`/users/*` — User management (protected)

Base path: `/users`

Routes in `user.route.js`:

- GET `/users/` — list all users (requires `isAuthenticated`)
- GET `/users/:id` — fetch user by id
- POST `/users/` — create user
 - Request body: `first_name`, `last_name`, `email`, `password`
 - Returns created user skeleton (password stored hashed)

- PUT /users/:id — update user fields
- DELETE /users/:id — delete user

/database/* — Database connections (lowdb-stored config)

Base path: /database

1. GET /database/ — list saved database connections
 - Returns array `db.data.dbConnections` (each entry contains connection fields and id)
2. POST /database/ — save a DB connection
 - Request body: any connection info (expected fields used by frontend; code accepts `req.body`), system adds `id` and `createdAt`.
 - Example response: returns the saved connection object.
3. DELETE /database/:id — delete a saved connection
 - Response: { success: true, message: "Connection deleted successfully" } or 404.
4. GET /database/tables — list tables for an external DB (connects using `knex-connection.js` service)
 - Query params likely include `connectionId` (caller should supply connection details).
 - Returns JSON list of table names on success.
5. GET /database/columns — list columns for a given table (expects `table` and `connection` info)
 - Returns list of column names.
6. POST /database/test — test DB connection (uses `createDBConnection` service)
 - Request body: connection details; returns success or error.

Notes: Stored DB connections are read from `lowdb` (`db.data.dbConnections`). These aren't primary data storage; they're configs for importing parsed rows into external DBs.

/files/* — Processed & incoming files

Base path: /files

- GET /files/ — list all files recorded in `lowdb` (`db.data.files`)
 - Query param `mailbox` optionally filters by mailbox email.
 - Response: array of file metadata (id, filename, status, mailbox, savedPath, createdAt, etc.)
- GET /files/:id/preview — preview CSV (first `n` rows).
 - Response: { rows: [...], file: { ... } } or 404 if file missing.
 - Implementation uses `processor.previewCsv` (max rows 20 in code).
- POST /files/:id/process — process an individual saved file (requires `mappingId` in body)

- Request body:
- { "mappingId": "mapping-id-123" }
- Behavior:
 - Finds file in `db.data.files`
 - Finds mapping (`db.data.mappings`) by ID
 - Finds `dbConnection` referenced by mapping (if mapping requires DB insert)
 - Maps CSV columns to DB columns (`mapping.columnMap`; `mapping.hasHeader` indicates header rows)
 - Uses `processor.importMappedRows` to insert rows into external DB based on mapping
 - Moves processed files to processed folder and writes import logs into `db.data.imports` (or similar)
- Responses:
 - Success: import result (number of rows, etc.)
 - 404 if file or mapping not found
 - 500 on processing error

Important: The code expects CSV parsing to be synchronous (parser from `csv-parse/sync`). If you have large files, memory usage should be considered.

`/imports/*` — Import jobs

Base path: `/imports`

- GET `/imports/` — list import logs (returns `db.data.imports` or similar)
- POST `/imports/process` — **process import** (body expects `fileId`, `mappingId`, `truncate` optional)
 - Behavior: code `imports.controller.processImport` finds file, mapping, DB connection then runs CSV parse, mapping, and insert; stores an import log object (with `id`, `status`, etc.)
 - Returns: created import log or errors (404 for missing file OR mapping)
- POST `/imports/save-mapping-and-import` — combined endpoint to save a mapping and trigger import in one call (UI convenience)
 - Body likely contains mapping details + `fileId`.

`/imap/poll` — IMAP manual poll trigger

Base path: `/imap`

- `POST /imap/poll` — triggers IMAP poll (`imapCtrl.pollImap`) which scans configured mailboxes
 - Request body: none required (system reads mailboxes from lowdb)
 - Response: summary of processed items (how many emails scanned/processed)
 - The system also supports scheduled cron jobs to run the same service.

Note: most IMAP processing is also available via service `processMailWithImap` used by `/mail/fetch-mails`.

`/mail/fetch-mails` — ad-hoc IMAP run with supplied credentials

Base path: `/mail`

- `POST /mail/fetch-mails`
 - **Body:** supply credentials for a one-off fetch:
 - `{`
 - `"user": "someone@gmail.com",`
 - `"password": "appPasswordOrOAuth2AccessToken",`
 - `"host": "imap.gmail.com",`
 - `"port": 993,`
 - `"tls": true`
 - `}`
 - **Return:** `{ success: true, processed: <result> }` or `{ success: false, error: "message" }`.
 - This exists as a convenience endpoint to run the IMAP processor for arbitrary credentials. In production you'll likely use saved inbox entries and the `/imap/poll` flow.

`/inboxes/*` — mailbox management & gmail config

Base path: `/inboxes`

Endpoints and behavior:

- `GET /inboxes/` — list mailboxes stored in lowdb (`db.data.mailboxes`)
 - Returns mailbox metadata and aggregated stats (processed count, failed count).
- `POST /inboxes/` — create a new inbox entry
 - Body fields: `email`, `displayName`, `gmailConfig` (optional), `imapConfig` (optional)...
 - System creates `id` and stores record in `db.data.mailboxes`.
- `DELETE /inboxes/:id` — delete the mailbox record
- `GET /inboxes/:id` — read a single mailbox and its files
- `PUT /inboxes/:id` — update mailbox config

- GET /inboxes/:id/analytics — aggregated stats for the specific inbox (last 30 days etc.)
- GET /inboxes/:id/gmail-config — retrieves the stored gmailConfig for the inbox
- PUT /inboxes/:id/gmail-config — update gmail configuration (client_id, secret, refresh_token, redirect_uri)
- POST /inboxes/:id/gmail-config/test — test the mailbox Gmail connection using the stored config (connects to Gmail using the refresh_token to get access token + IMAP test)

Request/Response shapes: the controller reads/writes `db.data.mailboxes` and returns the saved object(s). `gmailConfig` typically includes the fields:

```
{
  "clientId": "...",
  "clientSecret": "...",
  "refreshToken": "...",
  "redirectUri": "https://developers.google.com/oauthplayground"
}
```

[/mappings/* — mapping definitions](#)

Base path: /mappings

- GET /mappings/ — list mappings (`db.data.mappings`)
- POST /mappings/ — create/save mapping
 - Body includes:
 - id (optional)
 - name
 - connectionId (db connection to insert to)
 - table (destination table)
 - columnMap — map of CSV header (or index) to DB column
 - hasHeader — boolean
 - truncate — boolean (truncate table)
- DELETE /mappings/:id — delete mapping

Mapping example

```
{
  "id": "map_123",
  "name": "Polling CSV",
  "connectionId": "db_1",
  "table": "polling_table",
  "hasHeader": true,

```

```

"columnMap": {
  "WardID": "ward_id",
  "Polling_station": "polling_station"
},
"truncate": false
}

```

[/archives/* — archive listing & download](#)

Base path: /archives

- GET /archives/ — list archived files (reads db.data.archivedFiles)
 - Returns array of archive metadata.
- GET /archives/:id/download — download archived file (serves file from storage/archives/). Returns 404 if missing on disk.

9. Data model (lowdb) & skeleton

Your backend uses **lowdb** (a JSON file) as the primary configuration & metadata store. Key collections observed in code:

- db.data.mailboxes — list of mailbox/inbox records (id, email, gmailConfig, status)
- db.data.files — saved files metadata (id, mailbox, savedPath, filename, status, date, createdAt)
- db.data.mappings — mapping definitions used to map CSV → DB columns
- db.data.dbConnections — saved external DB connection configs
- db.data.imports or db.data.importLogs — import job logs (controller code appends import logs)
- db.data.archivedFiles — archived/processed files metadata
- db.data.users — users for authentication (email, hashed password, id)

Sample skeleton

```

{
  "mailboxes": [],
  "files": [],
  "mappings": [],
  "dbConnections": [],
  "imports": [],
  "archivedFiles": [],
  "users": []
}

```

Lowdb notes

- The JSON file is created automatically if not present (see config/lowdb.js).

- Backup the lowdb JSON regularly and keep it out of version control.

10. Workflows — end-to-end behavior

A. Automatic inbox polling (recommended)

1. Cron job triggers `services/mail-imap.js`:
2. Service reads the configured `db.data.mailboxes`.
3. For each inbox:
 - Uses stored `GmailConfig` (refresh token) to obtain access token via Google OAuth (if Gmail) or plain IMAP creds if configured.
 - Connects to IMAP, fetches new emails since last run, filters attachments for supported types (CSV).
 - Downloads attachments to `uploads/<inbox-id>/...`, creates a file record in `db.data.files` (status pending).
 - If there's an auto-mapping and auto-import configured, the service can process the file right away using mapping.
4. After processing, file status is updated to `processed` or `failed`.
5. Files are moved to `uploads/{mail-box}/email` and a record appended to `db.data.imports`.

B. Manual import from UI

- User uploads or selects a file from `/files` preview.
- User chooses mapping from `/mappings` (or creates mapping).
- Frontend calls `POST /files/:id/process` or `POST /imports/process` to run import using selected mapping.
- Controller parses CSV -> maps columns -> uses `processor.importMappedRows` to write to external DB (if configured) and saves import log.

11. Deployment & production tips

Quick production checklist

- Ensure `.env` has secure credentials and that `LOWDB_FILE` points to a proper, backed-up location.
- Ensure `uploads/` and `storage/archives/` are on durable volumes with enough space.
- Run the server under process manager: `pm2`, `systemd`, or containerize.
- Use HTTPS in production. Put the backend behind a reverse proxy (nginx) or run via cloud load balancer.
- Rotate and revoke OAuth tokens when rotating keys.

12. Monitoring, logging & maintenance

Logging

- Winston logger (config under `config/winston.js`) writes logs to `logs/`. Ensure log rotation is setup (daily or size-based).
- Monitor logs for repeated authentication errors (Gmail token revoked) and IMAP connectivity issues.

Maintenance tasks

- Review `uploads/` and `storage/archives/` monthly. Archive older files to cold storage if needed.
- Backup `db/data.json` periodically.
- Run `npm audit` and update dependencies regularly.
- If many large imports fail, inspect `processor` logs and memory usage.

13. Troubleshooting checklist (quick actionable)

- **IMAP connection fails:**
 - Verify `gmailConfig.refreshToken` is valid (use OAuth playground to test).
 - Check Gmail security settings (for non-OAuth, app passwords or “less secure apps” setting).
 - Check network connectivity to `imap.gmail.com:993`.
- **Attachment not parsed:**
 - Confirm file saved path exists and readable.
 - Check mapping: column keys must match CSV headers or indexes accordingly.
- **Import fails to external DB:**
 - Validate `dbConnections` entry: host, user, password, database.
 - Run `POST /database/test` with connection payload to test connection.
- **UI returns 500 for protected endpoints:**
 - If `users` endpoints: ensure you included `Authorization: Bearer <token>` header obtained from `/auth/login`.
- **lowdb file corrupted:**
 - Restore `db/data.json` from backup and restart service.

14. Appendix

A. Full sample .env (put in root of backend)

```
PORT=4000
NODE_ENV=production
LOWDB_FILE=./db/data.json
UPLOADS_BASE=./uploads
# Gmail OAuth2
GMAIL_CLIENT_ID=360438430623-
mtca7i6q77piaqli18t5ong03lcpvoaa.apps.googleusercontent.com
GMAIL_CLIENT_SECRET=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
GMAIL_REDIRECT_URI=https://developers.google.com/oauthplayground
GMAIL_REFRESH_TOKEN=1//04Pd2O8t294DmCgYIARAAGASNWf-L9IrY...
```

B. lowdb example snippet (small)

```
{
  "mailboxes": [
    {
      "id": "inbox_1",
      "email": "some@gmail.com",
      "displayName": "Some Mailbox",
      "gmailConfig": {
        "clientId": "...",
        "clientSecret": "...",
        "refreshToken": "1//...",
        "redirectUri": "https://developers.google.com/oauthplayground"
      }
    }
  ],
  "files": [
    {
      "id": "file_1",
      "mailbox": "some@gmail.com",
      "filename": "polling.csv",
      "savedPath": "./uploads/...",
      "status": "pending",
      "date": "2025-10-10T12:00:00Z",
      "createdAt": "2025-10-10T12:00:01Z"
    }
  ],
  "mappings": [],
  "dbConnections": [],
  "archivedFiles": [],
  "imports": [],
  "users": []
}
```